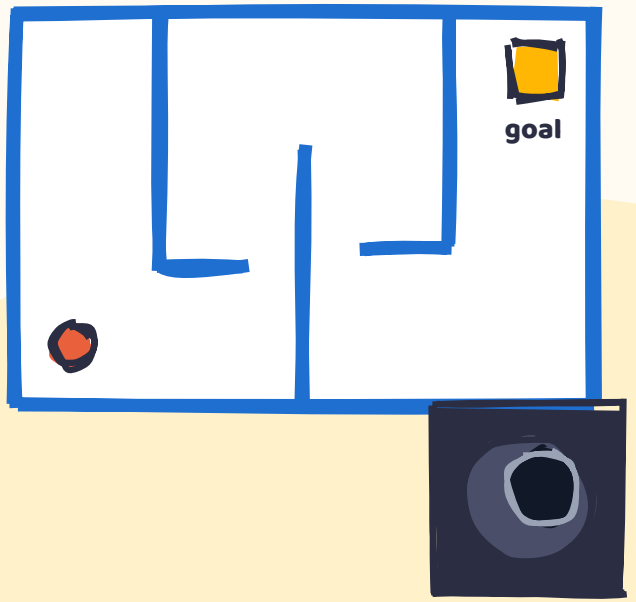
Junior Makers · Class 20

Joystick Maze Game

Your own controller, your own maze

Joystick | If + thresholds | Collisions | Level design



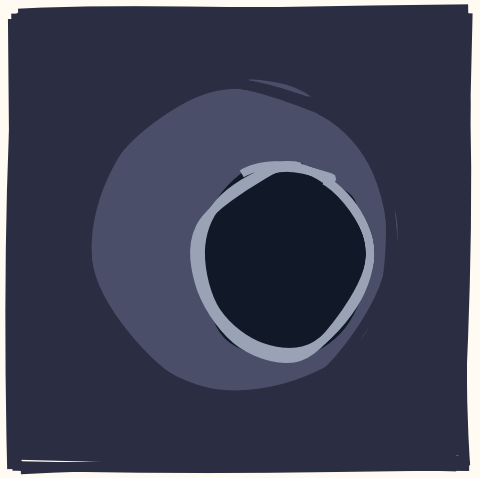
Did you know?
The first joysticks were made for aeroplanes. 'Joy stick' was pilot slang from the early 1900s.

Let's think...

How does a stick become numbers?

A tiny stick

You push a stick a little: the character walks. Push it all the way: it runs. Push it diagonally: it goes diagonally. How does a small plastic stick turn your thumb into **numbers** a computer understands?



Tiny buttons?

A camera?

Two knobs inside!

Today: you build a maze game in **mBlock** and control it with a **real joystick** on your Arduino.

Learning objectives

By the end of this class I can...

1	**Joystick numbers:** read a joystick's **X and Y** as numbers from 0 to 1023	**Proof:** our table of readings for centre, left, right, up, down
2	**Move with if:** use **if** blocks with **thresholds** to move a sprite	**Proof:** my sprite moves the way I push and stays still in the middle
3	**Collisions:** make walls send you back and a goal that **wins** with light and sound	**Proof:** touching a wall resets me; reaching the goal lights the LED
4	**Design & test:** design a maze that is **hard but fair** and improve it after testing	**Proof:** one change made after a friend played

Objective 1 · Joystick numbers · video

Video time!



How do games controllers work?

Fun Kids Learn · about 2 minutes · youtu.be/JfJQTqzSpFw

Watch for...

1. What is inside a controller that senses the **stick**?
2. How does the controller talk to the console?
3. What makes a controller **vibrate**?

Scan to watch

After the video: Mime holding a controller. Which thumb moves the character? Which presses the buttons?

Objective 1 · Joystick numbers

A joystick is two knobs at right angles

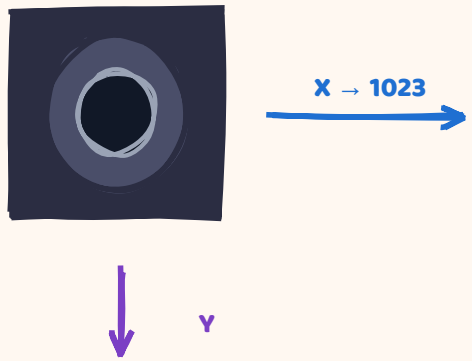
The idea

Inside are two **potentiometers** (knobs): one for **X** (left–right), one for **Y** (up–down). The Arduino reads each as a number from **0 to 1023**.

Stick in the middle: about **512, 512**. Push right: X goes up towards 1023. Push left: X goes down towards 0.

Secret button
Pressing the stick down is a hidden button (SW). We won't need it today, but try it later!

Example: a volume knob



Turning a speaker's volume knob changes how loud it is, smoothly. A joystick has two of these knobs joined to one stick, so one push can change both at once (diagonal!).

Objective 2 · Move with if

Thresholds: only move when pushed

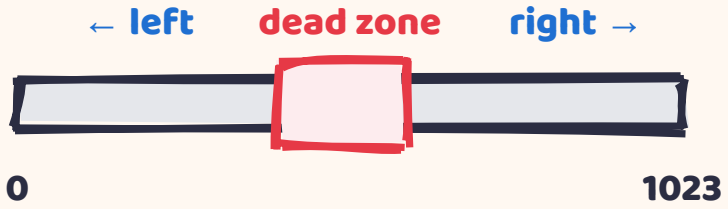
The idea

The middle is never exactly 512: it might be 508 or 517. If we moved whenever X > 512, the sprite would drift!

So we use **thresholds**:

- **if X > 600** → move right
- **if X < 400** → move left
- between 400 and 600 → **do nothing** (dead zone)

Example: a see-saw



A see-saw with two light children barely moves. It only tips when one side is clearly heavier. Our thresholds make the sprite move only when you clearly push.

Objective 3 · Collisions

Collisions: touching a colour

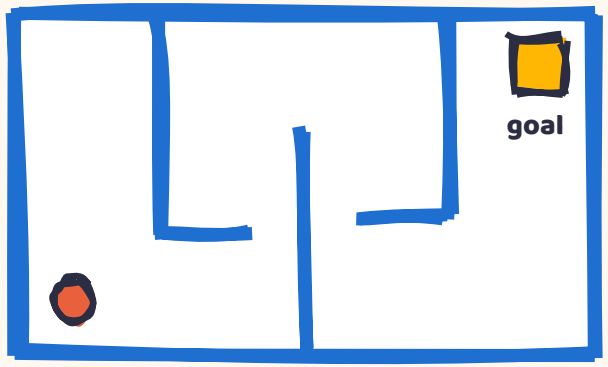
The idea

Draw the maze walls in **one colour** (e.g. blue) on the backdrop. Then:

- **if touching colour blue** → go back to the start, play a 'bonk' sound
- **if touching Goal** → you win! Light the LED, play a tune, stop

The computer checks these rules many times a second, inside the **forever** loop.

Example: electric buzz-wire game



At a fair, you guide a metal loop along a twisty wire. Touch the wire: BUZZ, start again! Our maze walls work exactly the same way.

Objective 4 · Design & test

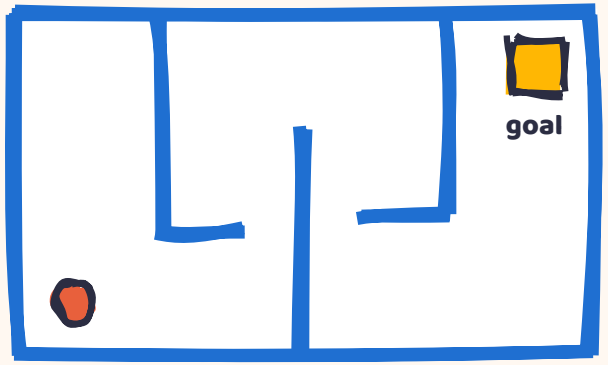
Hard but fair

The idea

A good maze is **challenging** but **fair**:

- Paths wide enough for the sprite (test it!)
- Start easy, get trickier near the goal
- One clear route, a few dead ends to explore
- No walls touching the start position!

Example: a school obstacle course



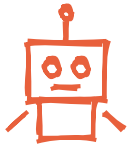
A sports-day obstacle course is fun when it's hard but possible. If one hurdle is taller than everyone, it isn't a challenge, it's just unfair.

Real world

Joysticks do serious jobs



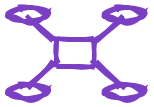
Aeroplanes
Pilots steer with a 'stick'



Cranes and diggers
Operators move huge arms with joysticks



Wheelchairs
Powered wheelchairs steer with a joystick



Drones
Two sticks control height, turning and direction

? Why does a wheelchair joystick need a dead zone even more than a game does?

Safety
A shaky hand must never make the chair move by accident.

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Human joystick
2. Our kit (per group)
3. Wire and read the joystick
4. Move the sprite with the stick
5. Walls, goal and a tries counter
6. Draw your maze and goal
7. Maze swap

Class game

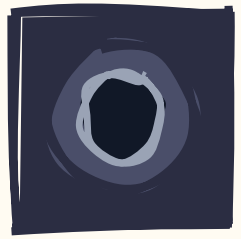
Human joystick

How to play

1. One student is the **sprite**, standing in a taped maze on the floor.
2. The **controller** holds a ruler like a joystick, standing behind them.
3. The sprite may only move the way the ruler tilts: tilted a little = small step, a lot = big step.
4. Ruler straight up = **don't move**!
5. Touch a wall? Back to the start.

Then discuss

When the ruler was nearly straight, did the sprite still wobble? How could we stop that?



Big idea

A 'do nothing' zone in the middle stops the sprite wobbling. Programmers call it a dead zone.

Build it

Our kit (per group)

Arduino Uno
Reads the joystick

Joystick module
X, Y and a button

LED + 220 Ω
Lights up when you win

Buzzer
Plays a sound

Laptop
mBlock 5 (live mode)

5 jumper wires
Female to male

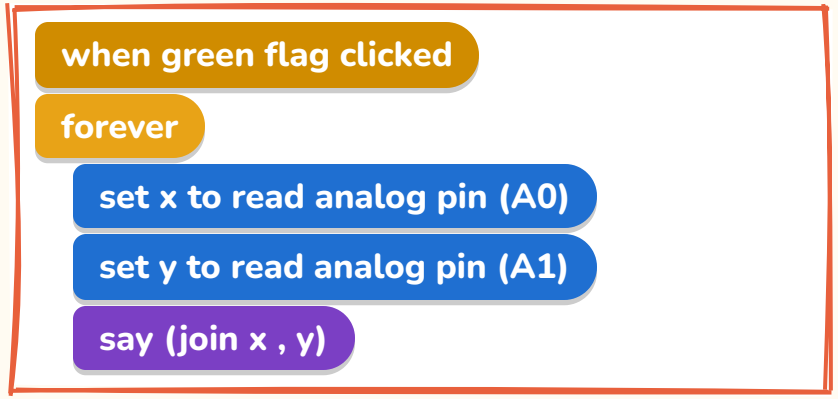
In mBlock, add the device **Arduino Uno**, connect, and switch to **Live** mode. Upload the mBlock firmware once if asked.

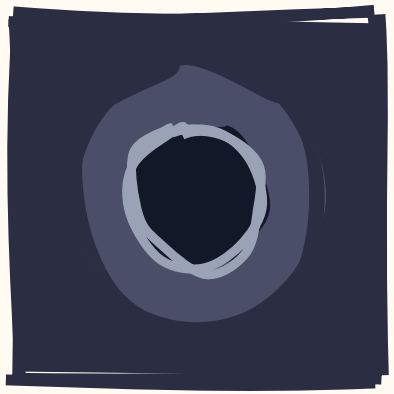
Live mode
The game runs on the laptop. The Arduino just sends the joystick numbers, live.

Objective 1 · Joystick numbers · Build step 1 of 4

Wire and read the joystick

Joystick / part	Arduino
GND	GND
+5V	5V
VRx	A0
VRy	A1
LED (+ 220 Ω)	pin 12
Buzzer +	pin 11





Push	X	Y
Centre		
Right / left		
Up / down		

Objective 2 · Move with if · Build step 2 of 4

Move the sprite with the stick

```
when green flag clicked
  go to x: -200 y: -140 (start)
  forever
    if read analog pin (A0) > 600 then change x by 3
    if read analog pin (A0) < 400 then change x by -3
    if read analog pin (A1) > 600 then change y by -3
    if read analog pin (A1) < 400 then change y by 3
```

Y backwards?

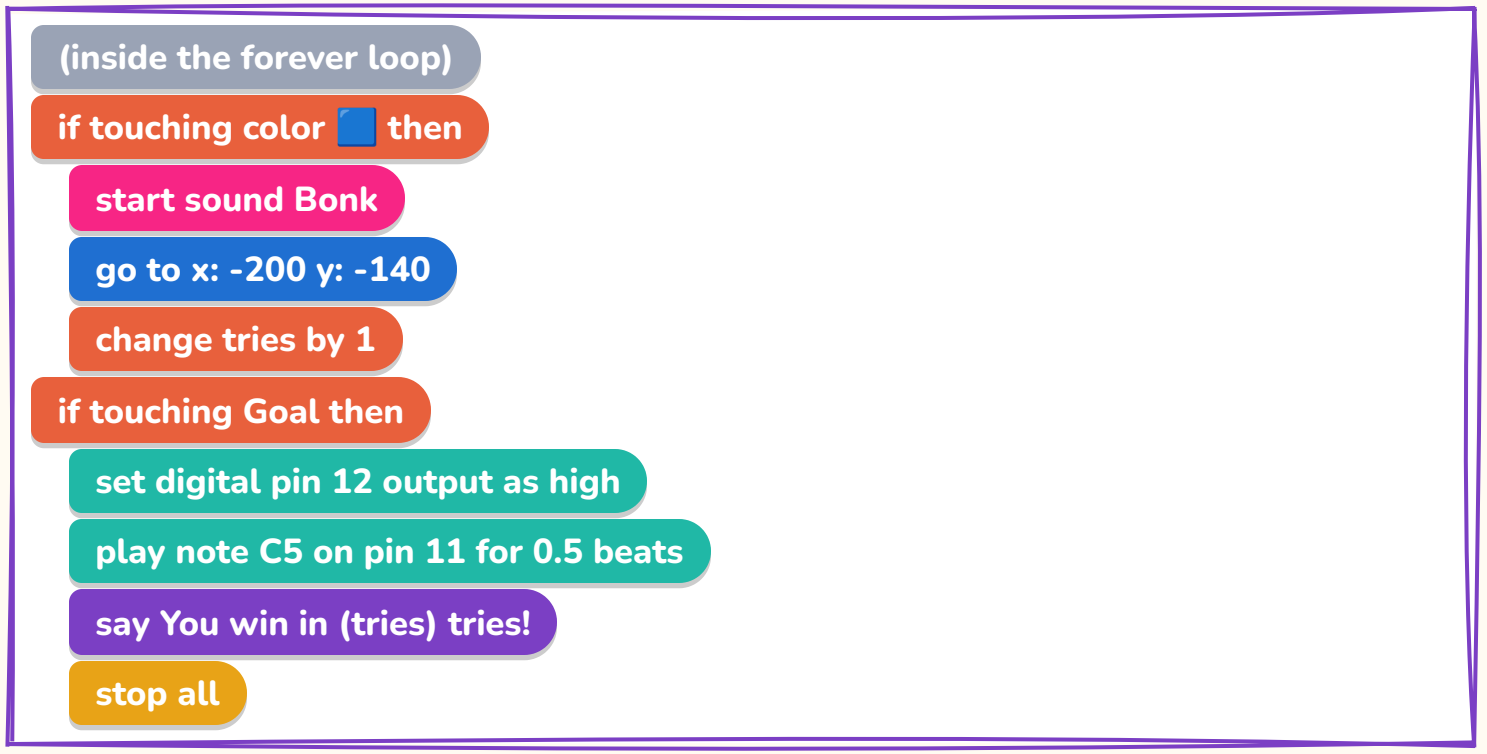
On many joysticks, pushing UP makes Y go DOWN. If your sprite moves the wrong way, swap the -3 and 3.

Speed

Change 3 to 5 for a faster sprite. Too fast and the maze gets frustrating!

Objective 3 · Collisions · Build step 3 of 4

Walls, goal and a tries counter



Variable

Make a variable called **tries**. Set it to 0 when the green flag is clicked. Fewer tries = better score!

Real hardware!

The LED and buzzer are on YOUR Arduino. The game reaches out of the screen into the real world.

Objective 4 · Build step 4 of 4

Draw your maze and goal

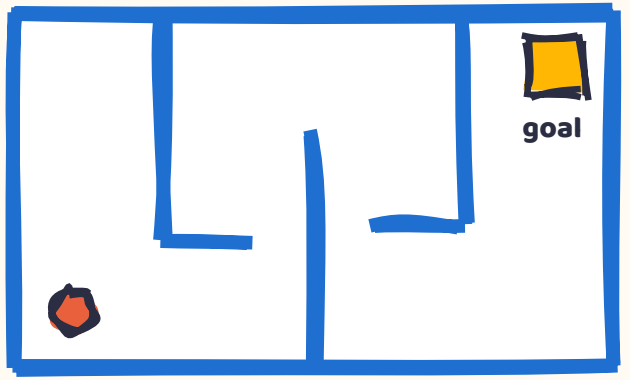
a Click the **Stage** → Backdrops → Paint. Use the rectangle tool, **blue** outline only.

b Make walls thick (at least 8 pixels) so the sprite can't slip through.

c Shrink your sprite (size 30%) so it fits the paths.

d Paint a new **Goal** sprite (a star or flag) and drag it to the end.

e Play it 3 times yourself. Can you always finish?



Check: Your maze has a start, a goal, and paths the sprite fits through.

Use ONE exact blue for walls. 'Touching colour' is fussy: a slightly different blue won't count.

Objective 4 · Design & test · playtest

Maze swap

Tester card	Answer
Tries to finish	
Time to finish	
Hardest bit	
Fair or unfair?	
One idea for the maker	

How to swap

Leave your game running. Move to another table. Play their maze **3 times** with their joystick. Fill in their tester card.

Makers

Read your cards. Change ONE thing to make your maze fairer or more fun.

Test like an engineer

Bug hunt

Bug	Likely cause	Did it happen?
Sprite drifts on its own	thresholds too close to 512	
Moves the wrong way up/down	Y is reversed	
Walls don't stop me	wall colour slightly different	
Reset at the very start	start position touches a wall	
Numbers don't change	not in Live mode / not connected	

Debug checklist

Make the sprite say x and y while you play. Seeing the numbers explains most bugs!

Fix one bug, test, then the next. Don't change everything at once.

Level up!

Maze challenges

Choose your challenge

Moving enemy: a sprite that glides back and forth across a path.

Timer: count seconds with a variable. Beat your best time!

Level 2: when you win, switch to a harder backdrop.

Speed control: push further = move faster (use the size of the reading).

Think like an engineer

Why do most games use thresholds instead of moving whenever the number changes a tiny bit?

Boss level

Use the joystick's push button (SW on pin 2) to give your sprite a 'dash' power.

Objective check

Can I really do it? Prove it.

01	Read out your joystick numbers for centre, left and right.	○ ○ ○
02	Show the if blocks with your thresholds. What is the dead zone?	○ ○ ○
03	Touch a wall, then reach the goal. What happens on the Arduino?	○ ○ ○
04	Tell us one change you made after your maze was tested.	○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Maze championship: fastest finish with the fewest tries on each table.

Show what you know

What did we learn?

- [x] A joystick is two knobs: X and Y, each 0 to 1023
- [x] Thresholds and a dead zone stop the sprite drifting
- [x] if touching colour → collision rules
- [x] Live mode lets the game control real LEDs and buzzers
- [x] Good mazes are hard but fair

Exit ticket: the sprite moves left when you don't touch the stick. What do you change?

At home

Find a joystick or knob at home (game controller, oven, radio). Which numbers might it send?



Next class:
Phase 2: NeoPixel Light Show to the Beat!